Глубокое обучение с подкреплением

Бонусный модуль 3: дополнительные темы и направления исследований

Курс Hugging Face Deep RL

Полная русскоязычная учебная версия на основе MDX

Источник: курс Hugging Face Deep RL (оригинальный текст на английском языке).

Содержание

1 Введение **2**

2 (Автоматическое) обучение по учебному плану в RL **2**
- 2.1 Дополнительные материалы . . . 4
- 2.2 Автор раздела . . . 5

3 Обобщающая способность в обучении с подкреплением **5**

4 Модельно-ориентированное обучение с подкреплением (MBRL) **5**
- 4.1 Простое определение . . . 5
- 4.2 Формальное определение . . . 6
- 4.3 Дополнительные материалы . . . 6
- 4.4 Автор раздела . . . 6

5 Офлайн- и онлайн-обучение с подкреплением **6**
- 5.1 Дополнительные материалы . . . 7
- 5.2 Автор раздела . . . 7

6 Трансформеры принятия решений (Decision Transformers) **8**
- 6.1 Как устроены трансформеры принятия решений . . . 8
- 6.2 Обучите свой первый Decision Transformer . . . 8
- 6.3 Дополнительные материалы . . . 8
- 6.4 Автор раздела . . . 8

7 Обучение с подкреплением на основе обратной связи от человека (RLHF) **8**
- 7.1 С чего начать изучение RLHF . . . 9
- 7.2 Дополнительная литература . . . 9
- 7.3 Автор раздела . . . 10

8 Языковые модели в RL **10**
- 8.1 Языковые модели содержат полезные знания для агентов . . . 10
- 8.2 Языковые модели и обучение с подкреплением . . . 11
- 8.3 Дополнительные материалы . . . 12
- 8.4 Автор раздела . . . 12

9 Знакомство с Unreal Learning Agents **12**
- 9.1 Случай 1: для начинающих и незнакомых с Unreal Engine . . . 13
- 9.2 Случай 2: для знакомых с Unreal Engine . . . 13

10 Godot RL Agents **13**
- 10.1 Создание собственной RL-среды с Godot RL Agents . . . 14
- 10.2 Автор раздела . . . 20

11 Введение в документирование RL-систем **20**
11.1 Зачем нужны отчёты о вознаграждениях (Reward Reports) . . . 21
11.2 Документирование изменений поведения во времени . . . 21
11.3 Участие в проекте . . . 21
11.4 Автор раздела . . . 21

12 Интересные среды для экспериментов **22**
12.1 DIAMBRA Arena . . . 22
12.2 MineRL . . . 23
12.3 Симулятор DonkeyCar . . . 24
12.4 StarCraft II . . . 25
12.5 Автор раздела . . . 25

13 Студенческие проекты **25**
13.1 Space Scavanger AI . . . 25
13.2 Neural Nitro . . . 26
13.3 Space War . . . 27
13.4 Трансформеры принятия решений для торговли . . . 27

1 Введение

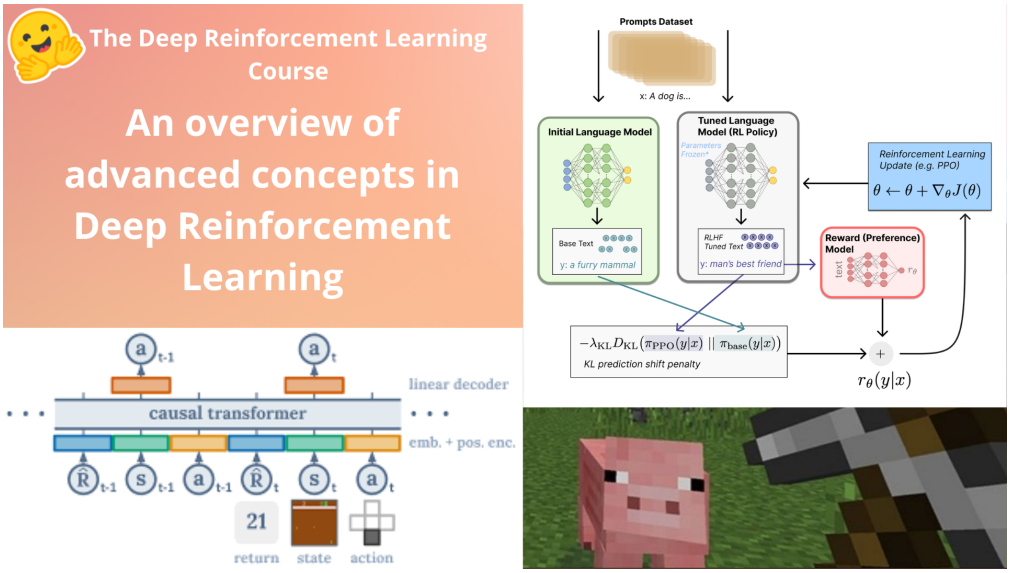


Рис. 1: Обзор бонусного модуля 3 курса Hugging Face Deep RL — источник: [оригинал изображения]

Поздравляем с завершением курса! **Теперь у вас есть прочная база знаний по глубокому обучению с подкреплением**. Но это лишь начало вашего пути: впереди множество интересных направлений. В этом необязательном модуле мы **собрали материалы для знакомства с различными концепциями и исследовательскими темами в RL**.

В отличие от других модулей, этот раздел подготовили сразу несколько участников команды Hugging Face. Для каждой темы указан её автор.

Звучит интересно? Тогда начнём!

2 (Автоматическое) обучение по учебному плану в RL

Хотя большинство изученных в этом курсе методов RL хорошо работают на практике, в некоторых случаях их недостаточно. Например, если:

- задача сложная и требует **постепенного освоения навыков** (скажем, прежде чем научиться преодолевать сложные препятствия, двуногий агент должен научиться стоять, затем

ходить и, возможно, прыгать. . .);

- среда изменяется, что влияет на сложность задачи, и требуется, чтобы агент был **устойчив** к таким изменениям.

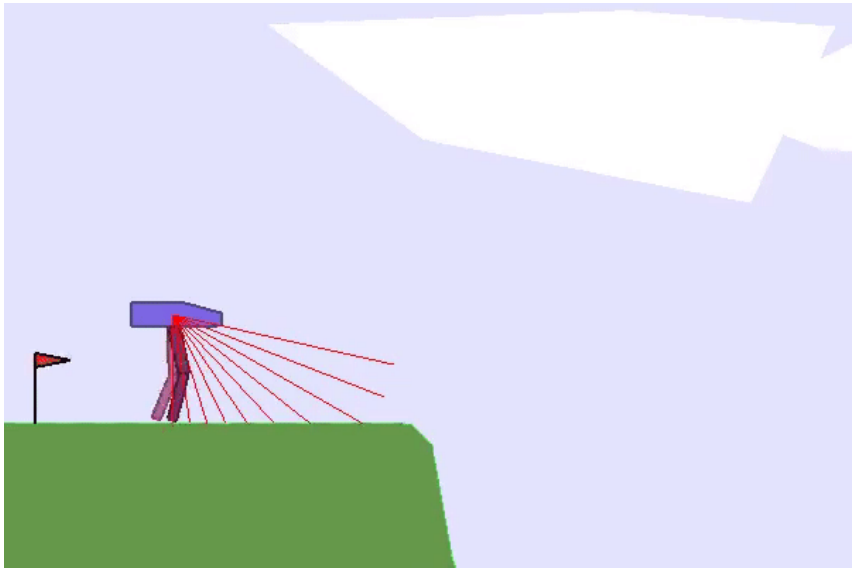


Рис. 2: Обучение двуногого агента ходьбе в TeachMyAgent: пример поэтапного обучения — источник: [оригинал изображения]

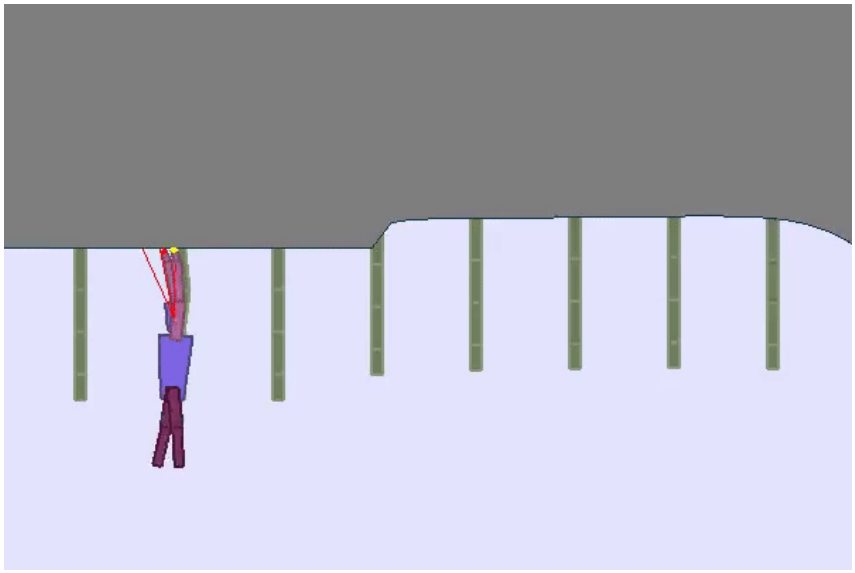


Рис. 3: Движущиеся препятствия в тестовой среде TeachMyAgent — источник: TeachMyAgent [оригинал изображения]

В таких ситуациях агенту нужно последовательно предлагать разные задачи, организованные так, чтобы он постепенно приобретал навыки. Этот подход называется **Curriculum Learning — обучением по учебному плану**; как правило, он предполагает вручную составленную последовательность задач. Например, можно изменять генерацию среды или начальные состояния либо использовать Self-Play, управляя сложностью предлагаемых агенту соперников.

Поскольку спроектировать такой учебный план не всегда просто, направление **Automatic Curriculum Learning (ACL) занимается методами, которые автоматически учатся выбирать и организовывать задачи для повышения эффективности RL-агента**. Портелас и соавторы предложили следующее определение ACL:

> ... семейство механизмов, которые автоматически адаптируют распределение обучающих данных, учась подбирать учебные ситуации в соответствии с возможностями RL-агентов.

Например, OpenAI использовала **рандомизацию домена (Domain Randomization)** — случайные изменения параметров среды — для обучения роботизированной руки собирать кубик Рубика.



Рис. 4: Рандомизация домена при обучении роботизированной руки собирать кубик Рубика — источник: OpenAI: роботизированная рука собирает кубик Рубика [оригинал изображения]

Наконец, вы можете сами проверить устойчивость агентов, обученных в TeachMyAgent: менять параметры среды или даже рисовать рельеф.

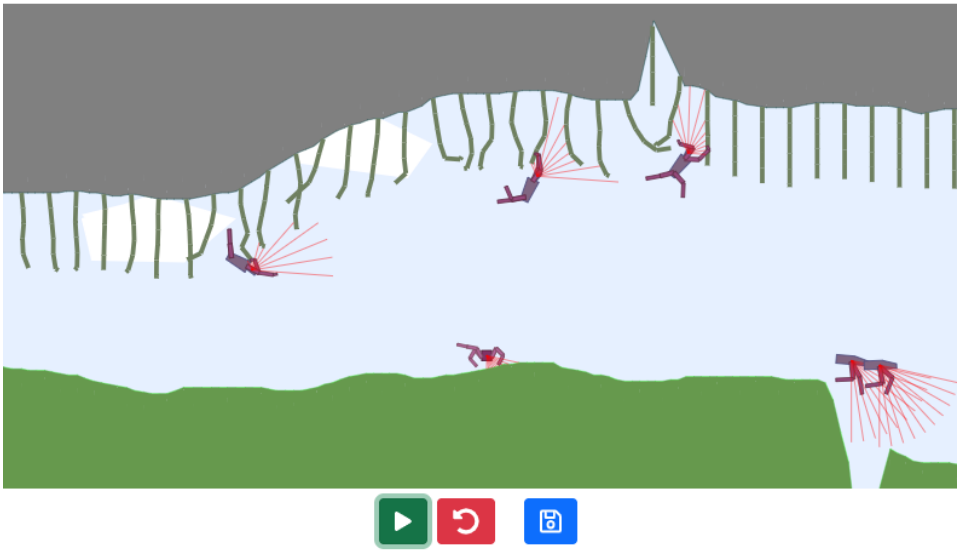


Рис. 5: Интерактивная среда TeachMyAgent с изменяемым рельефом — источник: https://huggingface.co/spaces/flowers-team/Interactive_DeepRL_Demo [оригинал изображения]

2.1 Дополнительные материалы

Для более подробного знакомства с темой рекомендуем следующие материалы:

Обзор направления

- Автоматическое построение учебного плана для Deep RL: краткий обзор

- Поэтапное обучение с подкреплением

Современные методы

- Эволюция учебных планов через проектирование сред на основе сожаления
- Обучение по учебному плану с ограниченным оптимальным транспортом
- Приоритетное повторное использование уровней

2.2 Автор раздела

Автор раздела: Клеман Ромак (Clément Romac).

3 Обобщающая способность в обучении с подкреплением

Обобщающая способность играет важнейшую роль в обучении с подкреплением. Хотя **алгоритмы RL могут хорошо работать в контролируемых средах**, реальный мир создаёт **особые трудности из-за нестационарности и отсутствия заранее заданных границ возможных ситуаций**.

Поэтому для применения RL в реальном мире принципиально важно разрабатывать алгоритмы, устойчивые к изменению условий, способные переносить знания на новые, но похожие задачи и адаптироваться к ранее неизвестным ситуациям.

Если вы хотите глубже изучить эту исследовательскую тему, рекомендуем начать со следующих материалов:

- Обобщающая способность в RL: обзор Роберта Кирка: в этом подробном обзоре даётся **полезное введение в понятие обобщающей способности в RL**, поэтому он отлично подходит для начала изучения темы.
- Улучшение обобщения в RL с помощью вложений сходства политик

4 Модельно-ориентированное обучение с подкреплением (MBRL)

Модельно-ориентированное обучение с подкреплением отличается от подходов без модели прежде всего тем, что агент изучает *модель динамики*. Однако это отличие существенно влияет на дальнейший процесс принятия решений.

Обычно такая модель описывает переходы между состояниями среды: $s_{t+1} = f_\theta(s_t, a_t)$. Кроме того, в этом подходе можно применять модели обратной динамики (переход от состояния к действию) и модели вознаграждения (прогнозирование награды).

4.1 Простое определение

- Агент многократно пытается решить задачу, **накапливая данные о состояниях и действиях**.
- На основе этих данных агент создаёт средство для рассуждений об окружающем мире — *модель динамики*.
- С помощью модели динамики агент **решает, как действовать, прогнозируя будущее**.
- Выполняя действия, **агент собирает новые данные, улучшает модель и, в идеале, повышает качество последующих действий**.

4.2 Формальное определение

В модельно-ориентированном обучении с подкреплением (MBRL) агент взаимодействует со средой, **изучает модель этой среды**, а затем **использует её для управления (принятия решений)**.

В частности, агент действует в марковском процессе принятия решений (MDP) с функцией перехода $s_{t+1} = f(s_t, a_t)$ и получает награду на каждом шаге $r(s_t, a_t)$. На основе собранного набора данных $D := s_i, a_i, s_{i+1}, r_i$ агент обучает модель $s_{t+1} = f_\theta(s_t, a_t)$, **минимизируя отрицательное логарифмическое правдоподобие наблюдаемых переходов**.

Мы используем метод управления с прогнозирующей моделью по выборкам (MPC). Обученная модель динамики позволяет максимизировать ожидаемое вознаграждение на конечном, рекурсивно прогнозируемом горизонте τ, рассматривая набор действий, выбранных из равномерного распределения $U(a)$ (см. работу, работу или работу).

4.3 Дополнительные материалы

Для более подробного изучения MBRL рекомендуем следующие материалы:

- Публикация в блоге об отладке MBRL.
- Обзорная научная статья по MBRL.

4.4 Автор раздела

Автор раздела: Нейтан Ламберт (Nathan Lambert).

5 Офлайн- и онлайн-обучение с подкреплением

Глубокое обучение с подкреплением (RL) — подход **к созданию агентов, принимающих решения**. Агент старается изучить оптимальное поведение (политику), взаимодействуя со средой **методом проб и ошибок и получая вознаграждение как единственный сигнал обратной связи**.

Цель агента — **максимизировать суммарное вознаграждение**, называемое возвратом (return). В основе RL лежит *гипотеза о вознаграждении*: любую цель можно описать как **максимизацию математического ожидания суммарного вознаграждения**.

Агенты глубокого обучения с подкреплением **обучаются на пакетах опыта**. Но откуда берутся эти данные?

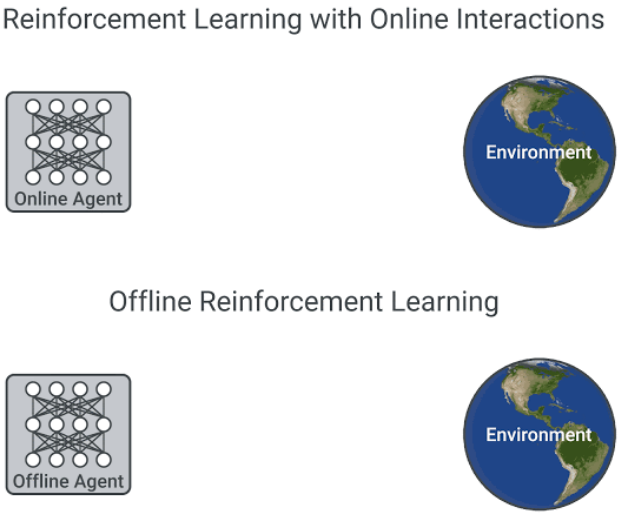


Рис. 6: Сравнение офлайн- и онлайн-обучения с подкреплением — источник: этот материал [оригинал изображения]

- При *онлайн-обучении с подкреплением*, которое мы изучали на протяжении курса, агент **непосредственно собирает данные**: получает пакет опыта, **взаимодействуя со средой**, а затем сразу использует его для обновления политики (или сохраняет опыт в буфере воспроизведения для дальнейшего обучения).

Однако это означает, что нужно либо **обучать агента непосредственно в реальном мире, либо иметь симулятор**. Если симулятора нет, его придётся построить. Это может быть сложно (как достоверно воспроизвести всю сложность реального мира?), дорого и небезопасно: агент способен использовать ошибки симулятора для получения преимущества.

- При *офлайн-обучении с подкреплением*, напротив, агент **использует только данные, собранные другими агентами или полученные из демонстраций людей**. Во время обучения он **не взаимодействует со средой**.

Процесс выглядит следующим образом:

- **Создайте набор данных** с помощью одной или нескольких политик и/или действий человека.
- **Обучите политику методом офлайн-RL**, используя этот набор данных.

У этого подхода есть недостаток: *проблема контрфактических запросов*. Что делать, если агент **решит выполнить действие, для которого в наборе данных нет примеров**? Например, он захочет повернуть направо на перекрёстке, но соответствующая траектория отсутствует.

Для этой проблемы существуют решения. Если хотите глубже изучить офлайн-обучение с подкреплением, можете посмотреть это видео.

5.1 Дополнительные материалы

Для более подробного знакомства с темой рекомендуем следующие материалы:

- Офлайн-обучение с подкреплением: лекция Сергея Левина
- Офлайн-обучение с подкреплением: руководство, обзор и открытые проблемы

5.2 Автор раздела

Автор раздела: Томас Симонини (Thomas Simonini).

6 Трансформеры принятия решений (Decision Transformers)

Модель Decision Transformer представлена в работе «Decision Transformer: Reinforcement Learning via Sequence Modeling» Чена и соавторов. Она рассматривает обучение с подкреплением как задачу условного моделирования последовательностей.

Основная идея заключается в следующем. Вместо того чтобы обучать политику методами RL (например, аппроксимировать функцию ценности, подсказывающую, какое действие максимизирует возврат), **мы используем алгоритм моделирования последовательностей — Transformer**. Получив желаемое значение возврата, предыдущие состояния и действия, он генерирует будущие действия, направленные на достижение этого возврата. Это авторегрессионная модель, обусловленная целевым возвратом, предшествующими состояниями и действиями.

Это существенное изменение парадигмы RL: вместо традиционных алгоритмов используется генеративное моделирование траекторий, то есть совместного распределения последовательностей состояний, действий и наград. Таким образом, Decision Transformer не максимизирует возврат напрямую, а генерирует последовательность будущих действий, которая должна привести к заданному возврату.

Команда Hugging Face Transformers интегрировала Decision Transformer — метод офлайн-RL — в библиотеку Transformers и Hugging Face Hub.

6.1 Как устроены трансформеры принятия решений

Чтобы лучше понять Decision Transformer, прочитайте наш материал «Знакомство с Decision Transformers на Hugging Face».

6.2 Обучите свой первый Decision Transformer

После знакомства с принципом работы Decision Transformer по нашему вводному материалу вы готовы самостоятельно обучить первую модель на офлайн-данных, чтобы она научила виртуального гепарда (HalfCheetah) бежать.

Начните с руководства: https://huggingface.co/blog/train-decision-transformers

6.3 Дополнительные материалы

Для дальнейшего изучения рекомендуем следующие материалы:

- Decision Transformer: RL через моделирование последовательностей
- Онлайн-вариант Decision Transformer

6.4 Автор раздела

Автор раздела: Эдвард Бичинг (Edward Beeching).

7 Обучение с подкреплением на основе обратной связи от человека (RLHF)

Обучение с подкреплением на основе обратной связи от человека (RLHF) — это **метод интеграции человеческих оценок в процесс оптимизации на основе RL**. Его применение связано с **трудностью формального моделирования человеческих предпочтений**.

Для многих задач даже при попытке записать формулу идеального результата предпочтения разных людей будут отличаться.

Обновление моделей **на основе измеряемых данных позволяет частично смягчить эти проблемы машинного обучения, неразрывно связанные с человеческими предпочтениями**.

7.1 С чего начать изучение RLHF

Чтобы приступить к изучению RLHF:

1. Прочитайте вводную статью: «Иллюстрированное введение в обучение с подкреплением по обратной связи от человека (RLHF)».
2. Посмотрите запись прямого эфира, в котором Нейтан объяснил основы RLHF и то, как эту технологию используют в современных инструментах машинного обучения, таких как ChatGPT. Большая часть лекции посвящена обзору связанных друг с другом ML-моделей. Рассматриваются основы обработки естественного языка, RL и применение RLHF к большим языковым моделям. В завершение обсуждаются открытые вопросы в RLHF.

Видеолекция: смотреть на YouTube

3. Прочитайте и другие статьи по теме, например «Продолжение дискуссии о закрытых API и открытом коде: RLHF, ChatGPT и преимущества данных». Если знаете интересные публикации, расскажите нам о них!

7.2 Дополнительная литература

Примечание: этот текст взят из упомянутой выше статьи об RLHF. Ниже перечислены основные научные публикации по RLHF на момент написания материала. Направление стало широко известно с развитием Deep RL (примерно с 2017 года), а затем расширилось в связи с применением больших языковых моделей в крупных технологических компаниях. Начнём с исследований, предшествовавших акценту на языковых моделях:

- TAMER: ручное обучение агента с помощью оценочного подкрепления (Knox и Stone, 2008): предложен агент, для которого люди последовательно оценивают выполненные действия, позволяя обучить модель вознаграждения.
- Интерактивное обучение на зависящей от политики обратной связи человека (MacGlashan и соавт., 2017): предложен актор-критик COACH, в котором положительные и отрицательные оценки людей используются для настройки функции преимущества.
- Глубокое обучение с подкреплением на основе человеческих предпочтений (Christiano и соавт., 2017): RLHF применяется к предпочтениям между траекториями в играх Atari.
- Deep TAMER: интерактивная настройка поведения агента в высокоразмерных пространствах состояний (Warnell и соавт., 2018): развитие TAMER, использующее глубокую нейросеть для прогнозирования вознаграждения.

Ниже — подборка исследований, демонстрирующих эффективность RLHF для языковых моделей:

- Дообучение языковых моделей на основе человеческих предпочтений (Zieglar и соавт., 2019): раннее исследование влияния обучения вознаграждению на четыре конкретные задачи.
- Обучение суммаризации текста на основе обратной связи человека (Stiennon и соавт., 2020): применение RLHF к краткому изложению текстов. Продолжение — Рекурсивное реферирование книг с обратной связью от человека (команда OpenAI Alignment, 2021).
- WebGPT: ответы на вопросы с помощью браузера и обратной связи человека (OpenAI, 2021): RLHF для обучения агента навигации по веб-страницам.
- InstructGPT: Обучение языковых моделей следованию инструкциям с помощью обратной связи человека (команда OpenAI Alignment, 2022): применение RLHF к языковой модели общего назначения (публикация в блоге об InstructGPT).

- GopherCite: Обучение языковых моделей подкреплять ответы проверенными цитатами (Menick и соавт., 2022): RLHF используется, чтобы языковая модель давала ответы с конкретными ссылками на источники.
- Sparrow: Улучшение согласования диалоговых агентов с помощью целевых оценок людей (Glaese и соавт., 2022): дообучение диалогового агента с использованием RLHF.
- ChatGPT: оптимизация языковых моделей для диалога (OpenAI, 2022): обучение универсального разговорного помощника с использованием RLHF.
- Законы масштабирования для чрезмерной оптимизации модели вознаграждения (Gao и соавт., 2022): исследуются закономерности масштабирования обученной модели предпочтений в RLHF.
- Обучение полезного и безопасного помощника с использованием RLHF (Anthropic, 2022): подробное описание обучения языковой модели-помощника с обратной связью человека.
- Красная команда против языковых моделей для снижения вреда: методы, масштабирование и выводы (Ganguli и соавт., 2022): подробное описание усилий по выявлению, измерению и снижению потенциально вредных ответов языковых моделей.
- Динамическое планирование в открытом диалоге с помощью обучения с подкреплением (Cohen и соавт., 2022): RL используется для улучшения разговорных навыков диалогового агента.
- (Не) подходит ли обучение с подкреплением для обработки естественного языка? Тестовые задачи, базовые методы и компоненты оптимизации языковой политики (Ramamurthy, Ammanabrolu и соавт., 2022): обсуждается пространство решений при создании открытых инструментов RLHF и предлагается алгоритм NLPO (Natural Language Policy Optimization) как альтернатива PPO.

7.3 Автор раздела

Автор раздела: Нейтан Ламберт (Nathan Lambert).

8 Языковые модели в RL

8.1 Языковые модели содержат полезные знания для агентов

Языковые модели (LM) демонстрируют впечатляющие способности при работе с текстом: от ответов на вопросы до пошаговых рассуждений. Кроме того, обучение на огромных текстовых корпусах позволяет им **сохранять знания разных типов, в том числе абстрактные представления о физических законах нашего мира** (например, что можно сделать с предметом и что произойдёт при его вращении).

Возникает естественный вопрос, который недавно начали активно изучать: могут ли такие знания помочь агентам, в том числе роботам, в выполнении повседневных задач? Хотя работы показали интересные результаты, предложенные агенты не обладали механизмом обучения. **Из-за этого они не могли адаптироваться к среде (например, исправлять неверные знания) или осваивать новые навыки**.

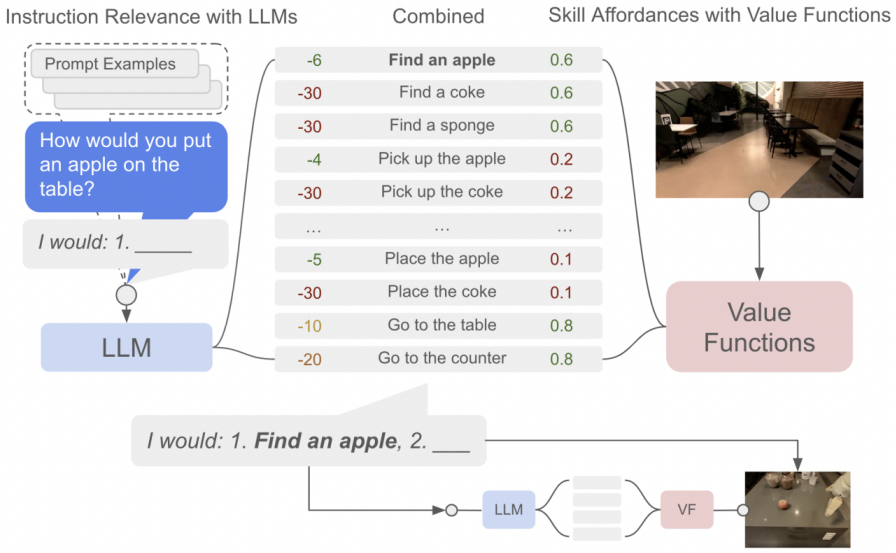


Рис. 7: Применение знаний языковой модели к возможностям роботизированных систем — источник: К полезным роботам: связь языка и возможностей роботов [оригинал изображения]

8.2 Языковые модели и обучение с подкреплением

Поэтому между языковыми моделями, предоставляющими знания о мире, и RL, которое позволяет согласовывать и корректировать эти знания через взаимодействие со средой, возможна полезная синергия. Это особенно важно для RL, поскольку в данной области часто используют постановку **tabula rasa**: агент изучает всё с нуля, что приводит к:

1) Неэффективному использованию обучающих примеров.
2) Поведению, которое кажется человеку неожиданным.

Одной из первых попыток стала работа «Привязка больших языковых моделей к среде посредством онлайн-обучения с подкреплением». Авторы решали задачу **адаптации и согласования языковой модели с текстовой средой при помощи PPO**. Было показано, что знания, уже содержащиеся в языковой модели, ускоряют адаптацию к среде (что открывает путь к RL с меньшими затратами выборки) и после согласования помогают лучше обобщать знания на новые задачи.

Видеодемонстрация: открыть оригинальное видео

Другое направление рассматривается в работе «Использование больших языковых моделей для направления предварительного обучения в RL». Языковая модель остаётся замороженной, но её знания применяются, чтобы **направлять исследование среды RL-агентом**. Это помогает агенту выбирать осмысленные для человека и потенциально полезные действия без необходимости постоянно привлекать человека во время обучения.

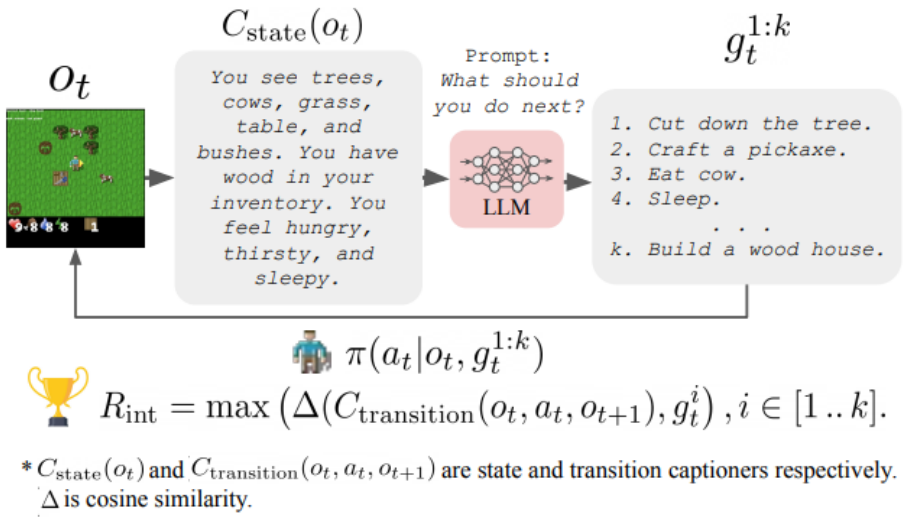


Рис. 8: Использование знаний языковой модели для исследования среды RL-агентом — источник: К полезным роботам: связь языка и возможностей роботов [оригинал изображения]

Пока эти результаты предварительны и имеют ряд ограничений: например, наблюдения агента приходится преобразовывать в текст перед передачей языковой модели, а взаимодействие с очень большими языковыми моделями требует значительных вычислительных ресурсов.

8.3 Дополнительные материалы

Для более подробного знакомства с темой рекомендуем следующие материалы:

- Google Research: робототехника в 2022 году и далее
- Предобученные языковые модели для интерактивного принятия решений
- Привязка больших языковых моделей к среде через онлайн-RL
- Направление предварительного обучения в RL с помощью языковых моделей

8.4 Автор раздела

Автор раздела: Клеман Ромак (Clément Romac).

9 Знакомство с Unreal Learning Agents

Learning Agents — плагин Unreal Engine (UE), позволяющий **обучать игровых персонажей на основе методов машинного обучения (ML) непосредственно в Unreal**.

Этот новый интересный плагин позволяет создавать собственные среды в Unreal Engine и обучать в них агентов.

Рассмотрим, как **начать работу и обучить виртуальный автомобиль управлению движением в среде Unreal Engine**.



Рис. 9: Unreal Engine Learning Agents: обучение агента управлению автомобилем — источник: Руководство по обучению автомобиля в Learning Agents [оригинал изображения]

9.1 Случай 1: для начинающих и незнакомых с Unreal Engine

Если вы ещё не знакомы с Unreal Engine, не переживайте! Чтобы начать использовать Learning Agents, предлагаем пройти два вводных курса:

1. Освойте основы: начните с курса «Ваш первый час в Unreal Engine 5». Он **даст базовые знания, необходимые для работы с Unreal**.
2. Познакомьтесь с Blueprints: изучите визуальную систему скриптов Unreal Engine. Этот видеокурс познакомит вас с данным инструментом.

Овладев основами, **вы готовы экспериментировать с Learning Agents**:

3. Получите общее представление о Learning Agents, прочитав этот обзор.
4. Научите автомобиль ездить с помощью обучения с подкреплением в Learning Agents.
5. Посмотрите руководство по имитационному обучению с плагином Learning Agents для Unreal Engine 5.3.

9.2 Случай 2: для знакомых с Unreal Engine

Если вы уже знакомы с Unreal Engine, можете сразу перейти к Learning Agents, воспользовавшись следующими руководствами:

1. Получите общее представление о Learning Agents, прочитав этот обзор.
2. Научите автомобиль ездить с помощью обучения с подкреплением в Learning Agents.
3. Посмотрите руководство по имитационному обучению с плагином Learning Agents для Unreal Engine 5.3.

10 Godot RL Agents

Godot RL Agents — пакет с открытым исходным кодом, который даёт разработчикам игр, исследователям ИИ и энтузиастам возможность **обучать сложному поведению неигровых персонажей (NPC) и агентов**.

Библиотека предоставляет:

- Интерфейс между играми, созданными в Godot Engine, и алгоритмами машинного обучения на Python.
- Обёртки для четырёх известных RL-фреймворков: StableBaselines3, CleanRL, Sample Factory и Ray RLlib.
- Поддержку агентов с памятью на основе LSTM или интерфейсов внимания.
- Поддержку *двумерных и трёхмерных игр*.
- Набор *датчиков ИИ*, расширяющих возможности агента по наблюдению игрового мира.
- Godot и Godot RL Agents **полностью бесплатны, имеют открытый код и распространяются по весьма свободной лицензии MIT**. Без скрытых условий и лицензионных отчислений.

Узнать больше о Godot RL Agents можно на странице GitHub или из статьи, представленной на семинаре AAAI 2022. Создатель библиотеки Эд Бичинг работает исследователем в Hugging Face.

Установка библиотеки проста: `pip install godot-rl`

10.1 Создание собственной RL-среды с Godot RL Agents

В этом разделе вы **научитесь создавать собственную среду в игровом движке Godot**, а затем реализуете контроллер ИИ, который учится играть с помощью глубокого обучения с подкреплением.

Сегодня мы создадим простую игру, **которая тем не менее демонстрирует многие возможности Godot Engine и библиотеки Godot RL Agents**. Затем вы сможете изучить примеры более сложных сред и способов поведения.

Наша среда называется Ring Pong. Это разновидность пинг-понга, где игровое поле имеет форму кольца, а ракетка перемещается по его окружности. **Цель — удерживать мяч внутри кольца, отбивая его ракеткой**.

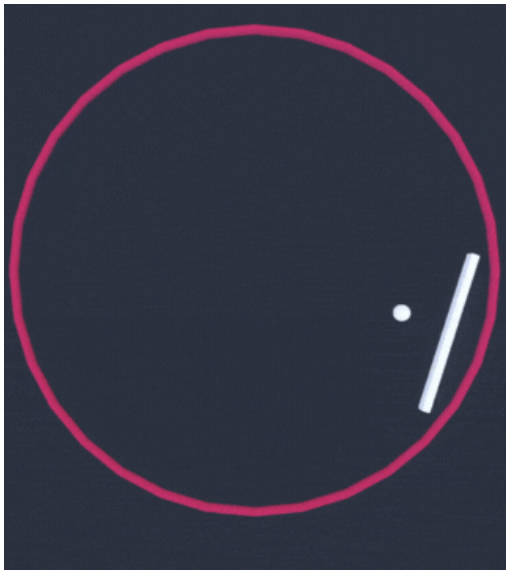


Рис. 10: Ring Pong: пользовательская игровая среда на Godot — источник: [оригинал изображения]

Установка игрового движка Godot

Игровой движок Godot — инструмент с открытым исходным кодом для **разработки видеоигр, программ и пользовательских интерфейсов**.

Godot Engine — многофункциональный кроссплатформенный игровой движок для создания 2D- и 3D-игр в единой среде. Он предоставляет большой набор стандартных инструментов,

поэтому разработчики **могут сосредоточиться на создании игр, не изобретая заново базовые механизмы**. Игры можно экспортировать практически одним нажатием для основных настольных ОС (Linux, macOS, Windows), мобильных платформ (Android, iOS) и веба (HTML5).

Мы проведём вас через реализацию агента, но, возможно, вам захочется лучше изучить сам Godot. Его документация весьма подробна, а на YouTube есть множество уроков. Также рекомендуем источники GDQuest, KidsCanCode и Bramwell.

Чтобы создавать игры в Godot, **сначала необходимо скачать редактор**. В материалах курса заявлена поддержка версии Godot 4.0 библиотекой Godot RL Agents.

Редактор можно скачать по следующим ссылкам:

- Windows
- Mac
- Linux

Загрузка стартового проекта

Мы предоставляем два варианта исходного кода:

- Стартовый проект: скачайте его и выполняйте шаги руководства.
- Готовая версия проекта для сравнения и поиска ошибок.

Чтобы загрузить проект, в Godot Project Manager нажмите **Import**, перейдите в каталог с файлами и выберите **project.godot**.

Если нажать F5 или запустить игру из редактора, можно играть в режиме человека. Одновременно запускается несколько копий игры, поскольку мы хотим ускорить обучение ИИ-агента с помощью параллельных сред.

Установка плагина Godot RL Agents

Плагин Godot RL Agents можно установить из репозитория GitHub или через встроенный каталог Godot AssetLib.

Сначала откройте AssetLib и выполните поиск по слову “rl.

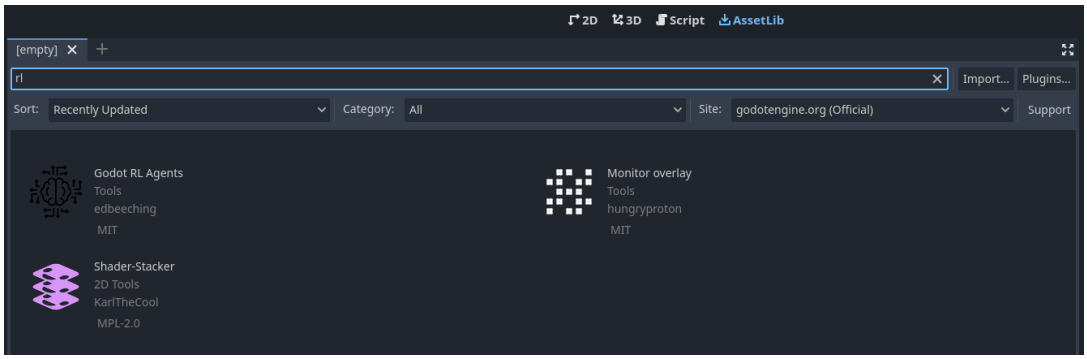


Рис. 11: Поиск плагина RL Agents в каталоге ресурсов Godot AssetLib — источник: [оригинал изображения]

Затем выберите Godot RL Agents, нажмите Download, снимите галочки с файлов LICENSE и README .md и нажмите Install.

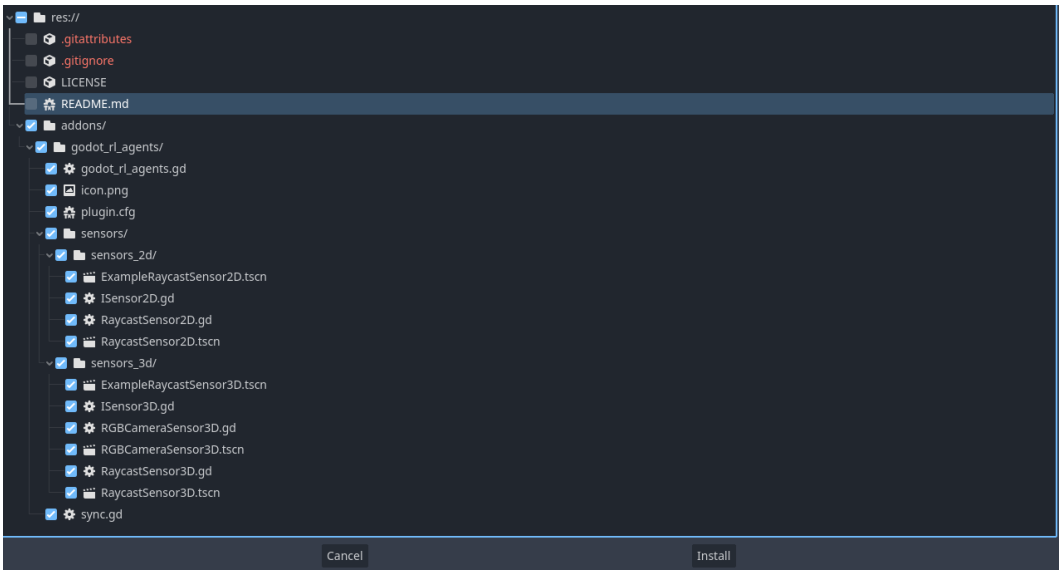


Рис. 12: Установка плагина RL Agents в Godot — источник: [оригинал изображения]

Теперь плагин Godot RL Agents загружен на компьютер. Откройте Project -> Project settings и включите дополнение:

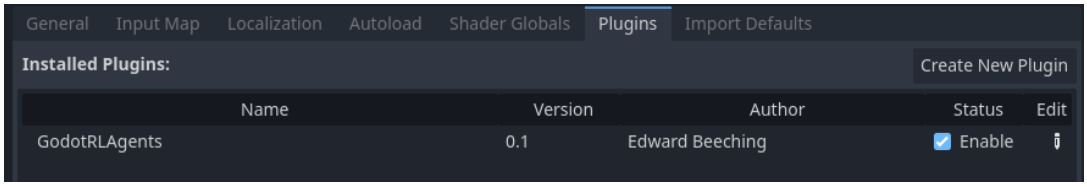


Рис. 13: Включение расширения RL Agents в настройках проекта Godot — источник: [оригинал изображения]

Добавление контроллера ИИ

Теперь добавим в нашу игру контроллер ИИ. Откройте сцену player.tscn: слева должно появиться дерево узлов примерно следующего вида:

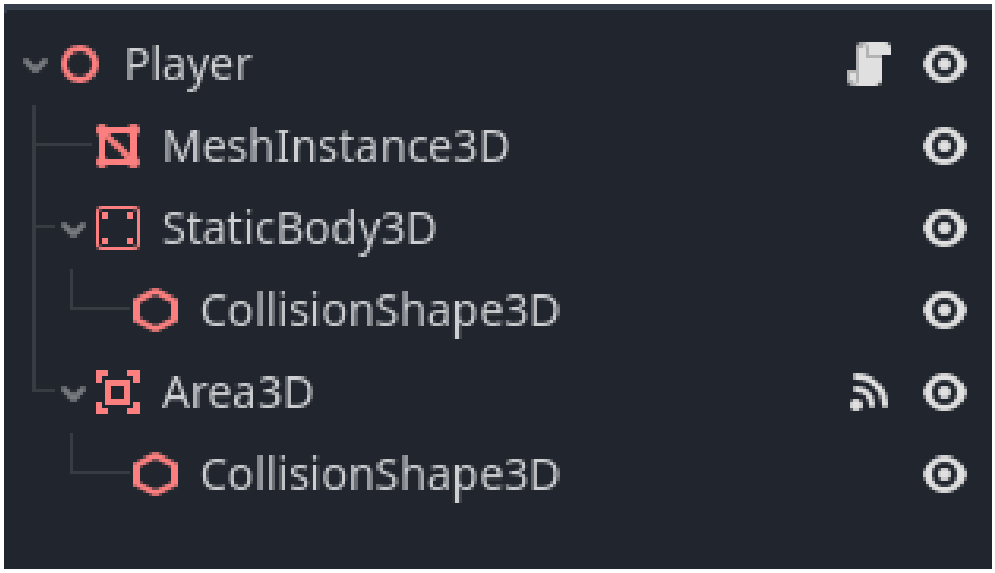


Рис. 14: Дерево сцены Godot перед добавлением контроллера ИИ — источник: [оригинал изображения]

Щёлкните правой кнопкой мыши по узлу **Player** и выберите **Add Child Node**. В списке узлов найдите AIController3D и создайте его.

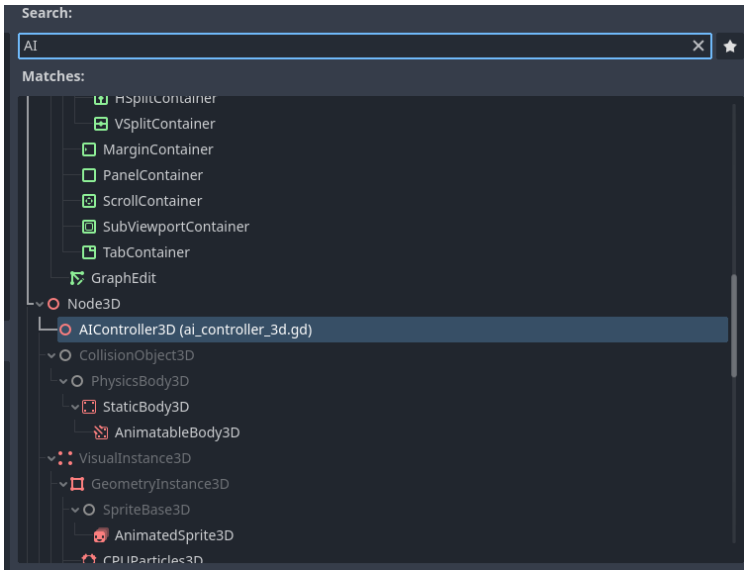


Рис. 15: Добавление дочернего узла AIController3D в Godot — источник: [оригинал изображения]

Узел AIController появится в дереве сцены. Рядом с ним будет значок скрипта: нажмите его, чтобы открыть связанный с контроллером код. Godot использует язык GDScript, синтаксически похожий на Python. Скрипт содержит методы, которые нужно реализовать для работы контроллера ИИ.

```
#-- Methods that need implementing using the "extend script" option in Godot --#
func get_obs() -> Dictionary:
    assert(false, "the get_obs method is not implemented when extending from ai_controller")
    return {"obs":[]}

func get_reward() -> float:
    assert(false, "the get_reward method is not implemented when extending from ai_controller")
    return 0.0

func get_action_space() -> Dictionary:
    assert(false, "the get get_action_space method is not implemented when extending from
    ai_controller")
    return {
        "example_actions_continous" : {
            "size": 2,
            "action_type": "continuous"
        },
        "example_actions_discrete" : {
            "size": 2,
            "action_type": "discrete"
        },
        }

func set_action(action) -> void:
    assert(false, "the get set_action method is not implemented when extending from ai_controller")
# -----------------------------------------------------------------------------#
```

Для реализации этих методов создадим класс, наследующий AIController3D. В Godot это делается легко и называется расширением класса (extending).

Щёлкните правой кнопкой мыши по узлу AIController3D, выберите “Extend Script и назовите новый скрипт `controller.gd`. В результате появится почти пустой файл следующего вида:

```
extends AIController3D

# Called when the node enters the scene tree for the first time.
func _ready():
```

```
    pass # Replace with function body.

# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta):
    pass
```

Теперь реализуем четыре отсутствующих метода: удалим этот шаблон и заменим его следующим кодом:

```
extends AIController3D

# Stores the action sampled for the agent's policy, running in python
var move_action : float = 0.0

func get_obs() -> Dictionary:
    # get the balls position and velocity in the paddle's frame of reference
    var ball_pos = to_local(_player.ball.global_position)
    var ball_vel = to_local(_player.ball.linear_velocity)
    var obs = [ball_pos.x, ball_pos.z, ball_vel.x/10.0, ball_vel.z/10.0]

    return {"obs":obs}

func get_reward() -> float:
    return reward

func get_action_space() -> Dictionary:
    return {
        "move_action" : {
            "size": 1,
            "action_type": "continuous"
        },
        }

func set_action(action) -> void:
    move_action = clamp(action["move_action"][0], -1.0, 1.0)
```

Теперь мы определили наблюдение агента — положение и скорость мяча в его локальной системе координат. Пространство действий тоже задано: одно непрерывное значение в диапазоне от -1 до +1.

Следующий шаг — обновить скрипт Player, чтобы использовать действия из AIController. Нажмите значок скрипта рядом с узлом Player и замените содержимое файла `Player.gd` следующим кодом:

```
extends Node3D

@export var rotation_speed = 3.0
@onready var ball = get_node("../Ball")
@onready var ai_controller = $AIController3D

func _ready():
    ai_controller.init(self)

func game_over():
    ai_controller.done = true
    ai_controller.needs_reset = true

func _physics_process(delta):
    if ai_controller.needs_reset:
        ai_controller.reset()
        ball.reset()
        return
```

```
    var movement : float
    if ai_controller.heuristic == "human":
        movement = Input.get_axis("rotate_anticlockwise", "rotate_clockwise")
    else:
        movement = ai_controller.move_action
    rotate_y(movement*delta*rotation_speed)

func _on_area_3d_body_entered(body):
    ai_controller.reward += 1.0
```

Теперь необходимо синхронизировать игру, запущенную в Godot, и нейросеть, обучаемую в Python. Для этого Godot RL Agents предоставляет специальный узел. Откройте сцену train.tscn, щёлкните правой кнопкой мыши по корневому узлу и выберите “Add Child Node. Найдите “sync и добавьте узел синхронизации Godot RL Agents Sync. Он обеспечивает обмен данными между Python и Godot по TCP.

Обучение можно запускать прямо в редакторе, предварительно запустив Python-процесс обучения командой `gdrl`.

В этом простом примере агент осваивает достаточно хорошую политику за несколько минут. Чтобы ускорить обучение, выберите узел Sync в сцене train: в инспекторе доступен параметр “Speed Up:

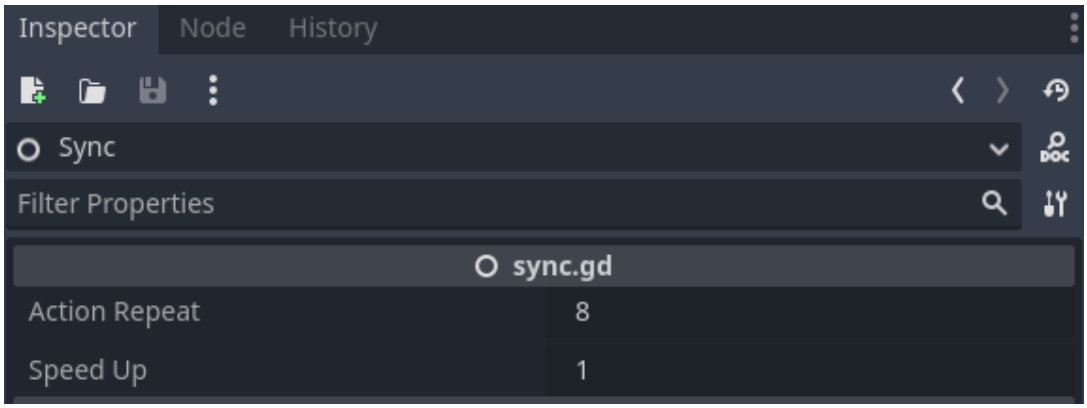


Рис. 16: Узел синхронизации Godot RL Agents: параметр Speed Up — источник: [оригинал изображения]

Попробуйте установить значение этого параметра равным 8. Это ускорит обучение и особенно пригодится в сложных средах, например в многопользовательском шутере от первого лица, который мы рассмотрим в следующей главе.

Экспорт модели

Справочная документация

На время отложим редактор Godot. Чтобы сохранить обученную модель, понадобится выполнить несколько команд в терминале.

В версии библиотеки Godot RL, описанной в исходном материале, доступна экспериментальная поддержка ONNX-моделей для Stable Baselines 3, RLlib и CleanRL.

Например, выберем Stable Baselines 3. Обучите агента при помощи примера SB3 (инструкция к скрипту), включив параметр `--onnx_export_path=model.onnx`.

Пример команды запуска:

```
cd <....> # go into this Godot project directory
python stable_baselines3_example.py --timesteps=100_000 --onnx_export_path=model.onnx --
    save_model_path=model.zip --save_checkpoint_frequency=20_000 --experiment_name=exp1
```

Если всё настроено правильно, в терминале появятся примерно следующие сообщения:

```
No game binary has been provided, please press PLAY in the Godot editor
waiting for remote GODOT connection on port 11008
```

> Если при запуске stable_baselines3_example возникает ошибка импорта “ImportError: cannot import name ‘export_model_as_onnx́from ‘godot_rl.wrappers.onnx.stable_baselines_export´, посмотрите решение в этом обсуждении.

Теперь вернитесь в редактор Godot и нажмите PLAY в правом верхнем углу. Откроется игровая сцена с обучающимся агентом, а в терминале начнут появляться метрики. Дождитесь завершения обучения. Если всё прошло успешно, в каталоге проекта Godot должен появиться файл `model.onnx`.

Использование обученной модели в игре

Теперь подключим обученную модель к игре!

В редакторе Godot найдите узел Sync в файле `train.tscn`:

- В выпадающем списке переключите режим управления на `Onnx Inference`.
- Укажите в параметре `Onnx Model Path` имя файла модели, в нашем случае `model.onnx`.

Для запуска игры в этом режиме нужна Mono-версия редактора Godot (вариант с поддержкой .NET), которую можно скачать с официального сайта. Также требуется установить .NET.

С большой вероятностью первая попытка завершится ошибкой. Ниже перечислены возможные проблемы и способы их устранения:

1. Ошибка `Invalid Call. Nonexistent function 'new' in base 'CSharpScript'`: решение.
2. Ошибки, связанные с `onnxruntime` на macOS: решение.

Что ещё можно сделать

Мы рассмотрели лишь малую часть возможностей Godot RL Agents. В библиотеке есть дополнительные пользовательские датчики и камеры, расширяющие информацию, которую получает агент. Посмотрите примеры, чтобы узнать больше!

Чтобы узнать об экспорте обученной модели в .onnx для запуска инференса непосредственно в Godot без Python-сервера, а также о других полезных параметрах обучения, изучите расширенное руководство по SB3.

10.2 Автор раздела

Автор раздела: Эдвард Бичинг (Edward Beeching).

11 Введение в документирование RL-систем

В этой дополнительной теме мы рассмотрим вопрос: **как отслеживать работу и документировать поведение мощных RL-агентов, которые мы обучаем в реальном мире и которые взаимодействуют с людьми?**

По мере того как системы машинного обучения всё сильнее влияют на современную жизнь, **растёт потребность в их документировании**.

Такая документация может описывать используемые обучающие данные — где они хранятся, когда были собраны, кто участвовал в сборе и т. д.; либо процесс оптимизации модели — архитектуру, метрики оценки, связанные статьи и многое другое.

Сегодня всё шире распространяются карточки моделей и паспорта наборов данных. Например, они доступны на Hub (см. документацию).

Открыв страницу популярной модели на Hub, можно узнать о процессе её создания.

Такие журналы, относящиеся к конкретной модели или набору данных, обычно заполняют во время их создания, но впоследствии не обновляют, когда модель становится частью развивающейся системы.

11.1 Зачем нужны отчёты о вознаграждениях (Reward Reports)

Системы RL по своей природе оптимизируются по сигналу вознаграждения во времени. Хотя понятие функции вознаграждения можно связать с хорошо изученными областями обучения с учителем (через функцию потерь), наше понимание того, как поведение систем машинного обучения изменяется со временем, пока ограничено.

Поэтому авторы предлагают *Reward Reports for Reinforcement Learning — отчёты о вознаграждениях в RL*. Название намеренно перекликается с известными работами *Model Cards for Model Reporting* и *Datasheets for Datasets*. Цель — предложить тип документации, ориентированный на **человеческие аспекты определения вознаграждения** и **системы обратной связи, изменяющиеся во времени**.

Опираясь на форматы документации карточек моделей и паспортов наборов данных, предложенные Митчелл и Гебру с соавторами, мы обосновываем необходимость отчётов Reward Reports для систем ИИ.

Reward Reports — постоянно обновляемые документы для предполагаемых внедрений RL, фиксирующие выбранные проектные решения.

Однако остаётся много вопросов о применимости такого подхода к разным RL-задачам, трудностях интерпретируемости систем и сходстве уже внедрённых систем обучения с учителем с последовательным принятием решений в RL.

Как минимум Reward Reports дают практикам RL возможность обсудить эти вопросы и приступить к поиску способов их решения.

11.2 Документирование изменений поведения во времени

Ключевой элемент документации для RL и ML-систем с обратной связью — *журнал изменений (change-log)*. Он фиксирует обновления со стороны разработчика (изменившиеся параметры обучения, данные и т. п.) и замеченные пользователями изменения (вредное поведение, неожиданные ответы и т. п.).

В дополнение к журналу предусмотрены события, которые должны запускать обновление отчёта и побуждают наблюдать за последствиями изменений.

11.3 Участие в проекте

Некоторые наиболее влиятельные RL-системы затрагивают множество заинтересованных сторон, но разрабатываются закрыто внутри частных компаний. Поскольку такие компании в значительной степени действуют без специального регулирования, ответственность за документирование часто ложится на общественность.

Если вы хотите принять участие, мы создаём публичные Reward Reports для популярных ML-систем на GitHub.

Для дальнейшего чтения обратитесь к статье о Reward Reports или посмотрите пример отчёта.

11.4 Автор раздела

Автор раздела: Нейтан Ламберт (Nathan Lambert).

12 Интересные среды для экспериментов

Здесь мы собрали интересные среды, в которых вы можете попробовать обучить своих агентов:

12.1 DIAMBRA Arena



Рис. 17: DIAMBRA Arena: аркадные среды для экспериментов с RL — источник: [оригинал изображения]

DIAMBRA Arena — программный пакет с набором качественных сред для исследований и экспериментов в области обучения с подкреплением. Он предоставляет стандартный интерфейс к популярным аркадным играм в эмуляторе, а его Python API полностью соответствует формату OpenAI Gym/Gymnasium, поэтому начать работу с ним легко.

Поддерживаются основные операционные системы (Linux, Windows, macOS); пакет легко установить через pip. Использование бесплатно, нужно лишь зарегистрироваться на официальном сайте.

Кроме того, в репозитории GitHub есть примеры основных сценариев использования, которые можно запустить за несколько шагов.

Основные возможности Все среды представляют собой эпизодические RL-задачи с дискретными действиями (кнопки геймпада) и наблюдениями в виде пикселей экрана, дополненными числовыми данными (значения RAM, например полосы здоровья или расположение персонажа на арене).

Поддерживаются одиночный режим (1P) и режим двух игроков (2P). Поэтому платформу удобно использовать для обычного RL, соревновательного многоагентного обучения, соревнований человека и агента, Self-Play, имитационного обучения и обучения с человеком в контуре (Human-in-the-Loop).

Доступные игры выбраны из числа популярных ретро-файтингов. Хотя их базовая механика схожа, каждая предлагает свои трудности: разный набор и число персонажей, комбинации приёмов, восстановление здоровья и т. д.

DIAMBRA Arena разработана с учётом совместимости с основными RL-библиотеками. Она предоставляет готовые интерфейсы для Stable Baselines 3 и Ray RLlib; также доступна устаревшая поддержка Stable Baselines. Примеры приведены в официальной документации и репозитории DIAMBRA Agents. По аналогии можно подключить и другие библиотеки.

Соревновательная платформа

DIAMBRA также предоставляет соревновательную платформу, полностью интегрированную с Hugging Face Hub: вы можете публиковать обученных агентов и соревноваться с другими разработчиками со всего мира в турнирах по видеоиграм!

На платформе есть публичная таблица лидеров, в которой пользователи ранжируются по лучшим результатам их агентов в разных средах.

В зависимости от результатов агента можно получать различные достижения.

Поступившие агенты оцениваются, а записи их эпизодов транслируются на канале DIAMBRA в Twitch.

Ссылки Чтобы начать работу с этой средой, воспользуйтесь следующими ресурсами:

- Официальная документация
- Соревновательная платформа
- GitHub
- Discord

12.2 MineRL

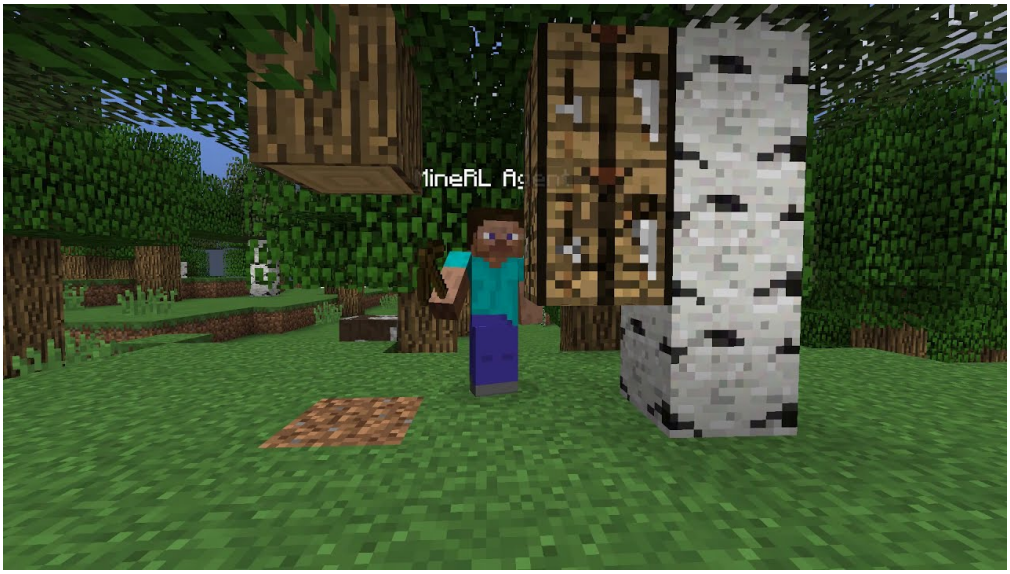


Рис. 18: MineRL: задачи обучения с подкреплением на основе Minecraft — источник: [оригинал изображения]

MineRL — библиотека Python с интерфейсом Gym для взаимодействия с игрой Minecraft и наборами данных, собранными из действий людей. Каждый год с использованием библиотеки проводятся соревнования. Подробнее — на сайте проекта.

Чтобы начать работу с этой средой, воспользуйтесь следующими ресурсами:

- Что такое MineRL?
- Первые шаги в MineRL
- Документация и руководства MineRL

12.3 Симулятор DonkeyCar

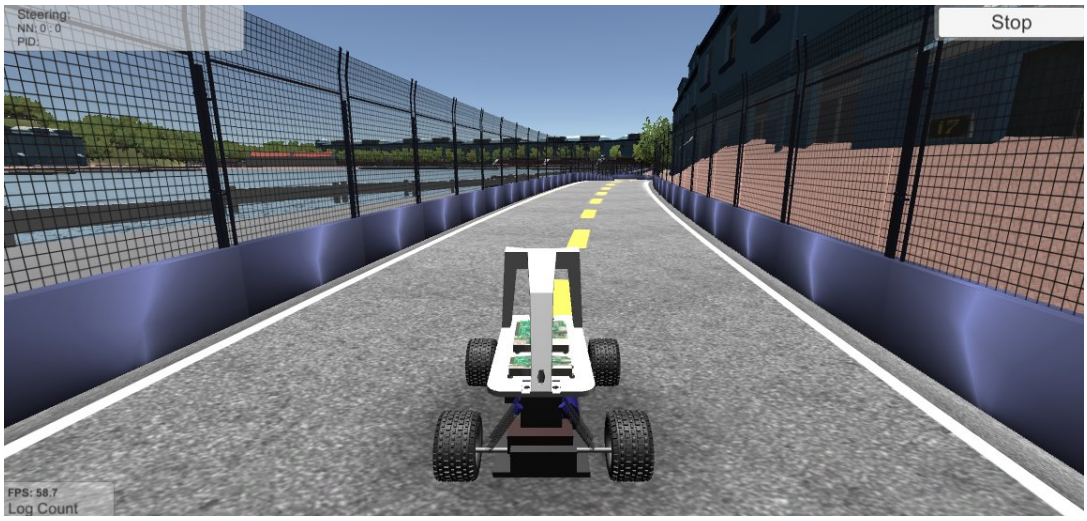


Рис. 19: Симуляционная среда DonkeyCar — источник: [оригинал изображения]

DonkeyCar — платформа автономного вождения для любительских радиоуправляемых автомобилей. Эта версия симулятора построена на Unity: используются физика и графика движка, а связь с процессом Python позволяет обученной модели управлять виртуальным автомобилем Donkey.

Чтобы начать работу с этой средой, воспользуйтесь следующими ресурсами:

- Документация симулятора DonkeyCar
- Плавное вождение: руководство Антонена Раффена, часть 1
- Плавное вождение: руководство Антонена Раффена, часть 2
- Плавное вождение: руководство Антонена Раффена, часть 3
- Предварительно обученные агенты:
  - https://huggingface.co/araffin/tqc-donkey-mountain-track-v0
  - https://huggingface.co/araffin/tqc-donkey-avc-sparkfun-v0
  - https://huggingface.co/araffin/tqc-donkey-minimonaco-track-v0

12.4 StarCraft II



Рис. 20: AlphaStar и StarCraft II как среда для исследования RL — источник: [оригинал изображения]

StarCraft II — известная *стратегия в реальном времени*. Компания DeepMind использовала эту игру для исследований в области глубокого RL, создав AlphaStar.

Чтобы начать работу с этой средой, воспользуйтесь следующими ресурсами:

- StarCraft Gym
- ИИ учится играть в StarCraft II: руководство по RL

12.5 Автор раздела

Автор раздела: Томас Симонини (Thomas Simonini).

13 Студенческие проекты

С момента запуска курса Deep Reinforcement Learning **многие студенты создали замечательные проекты, с которыми стоит познакомиться и в которых можно принять участие**.

Если вы разработали интересный проект, можете добавить его в этот список, создав pull request в репозитории GitHub.

Проекты **отсортированы по датам их публикации на этой странице**.

13.1 Space Scavanger AI

Этот проект представляет собой космическую игровую среду с нейросетевым ИИ.

ИИ обучается с помощью методов RL на основе фреймворков Unity ML-Agents и RLlib.

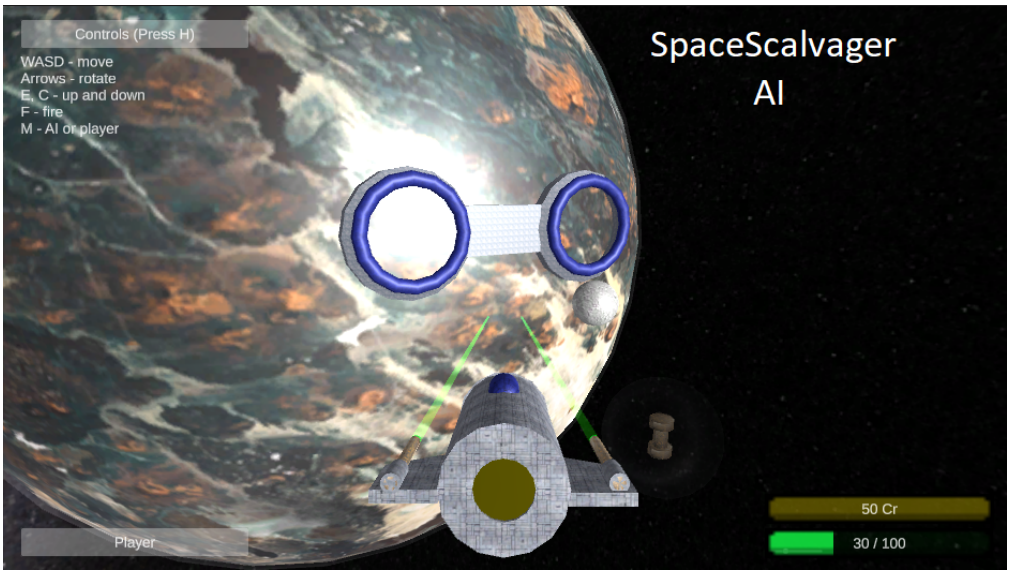


Рис. 21: Студенческий проект Space Scavanger AI — источник: [оригинал изображения]

Играть можно здесь: https://swingshuffle.itch.io/spacescalvagerai

Посмотреть проект Unity: https://github.com/HighExecutor/SpaceScalvagerAI

13.2 Neural Nitro

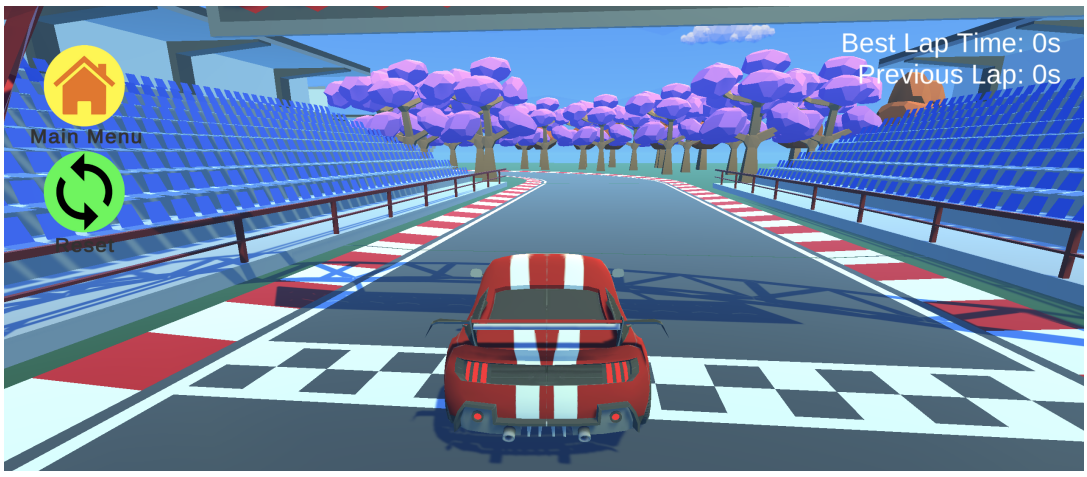


Рис. 22: Студенческий проект Neural Nitro: гоночная среда — источник: [оригинал изображения]

В этом проекте Sookeyy создал низкополигональную гоночную игру и научил автомобиль ездить.

Посмотреть демонстрацию: https://sookeyy.itch.io/neuralnitro

13.3 Space War



Рис. 23: Студенческий проект Space War — источник: [оригинал изображения]

В этом проекте Эрик Донг (Eric Dong) воссоздаёт в Pygame версию игры Space War Билла Сайлера (1985 год) и использует обучение с подкреплением для тренировки ИИ-агентов.

Проект пока находится в разработке!

Демонстрация

Версия для разработчиков (Dev/Edge):

- https://e-dong.itch.io/spacewar-dev

Стабильная версия:

- https://e-dong.itch.io/spacewar
- https://huggingface.co/spaces/EricofRL/SpaceWarRL

Публикации сообщества

Будет объявлено позже.

Другие ссылки

Исходный код: https://github.com/e-dong/space-war-rl
Блог автора: https://dev.to/edong/space-war-rl-0-series-introduction-25dh

13.4 Трансформеры принятия решений для торговли

В этом проекте студент исследовал обучение Decision Transformer для биржевой торговли. На первом этапе реализовано офлайн-обучение; в следующей версии планируется добавить онлайн-дообучение.

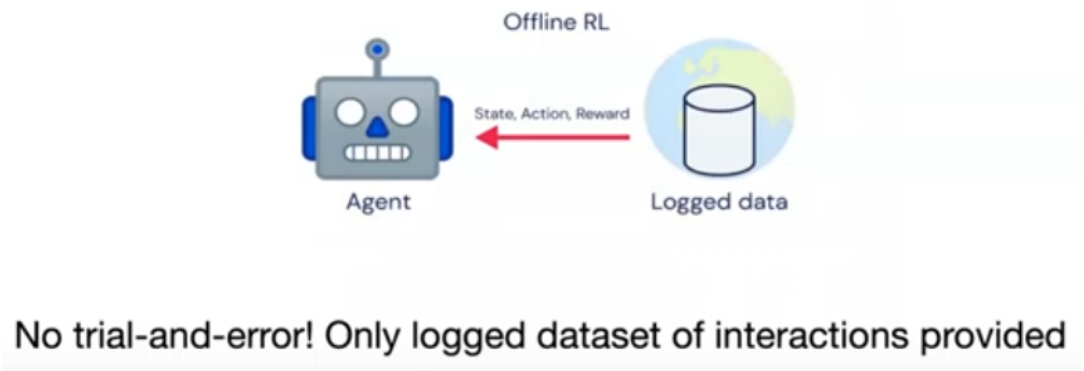


Рис. 24: Студенческий проект Decision Transformers for Trading — источник: Stanford CS25: Decision Transformer и моделирование последовательностей [оригинал изображения]

Исходный код: https://github.com/ra9hur/Decision-Transformers-For-Trading